Architecture complète des agents de DressMe

Qui est qui, qui parle à qui, et où intervient l'orchestrateur

Réponse à la question clé. L'orchestrateur n'intervient **que dans le chat de la page AI Stylist** : c'est la seule page qui appelle `/api/assistant/chat`. Toutes les autres pages appellent leur agent directement. Ce n'est donc pas un passage obligé du système : c'est une entrée de conversation en plus.

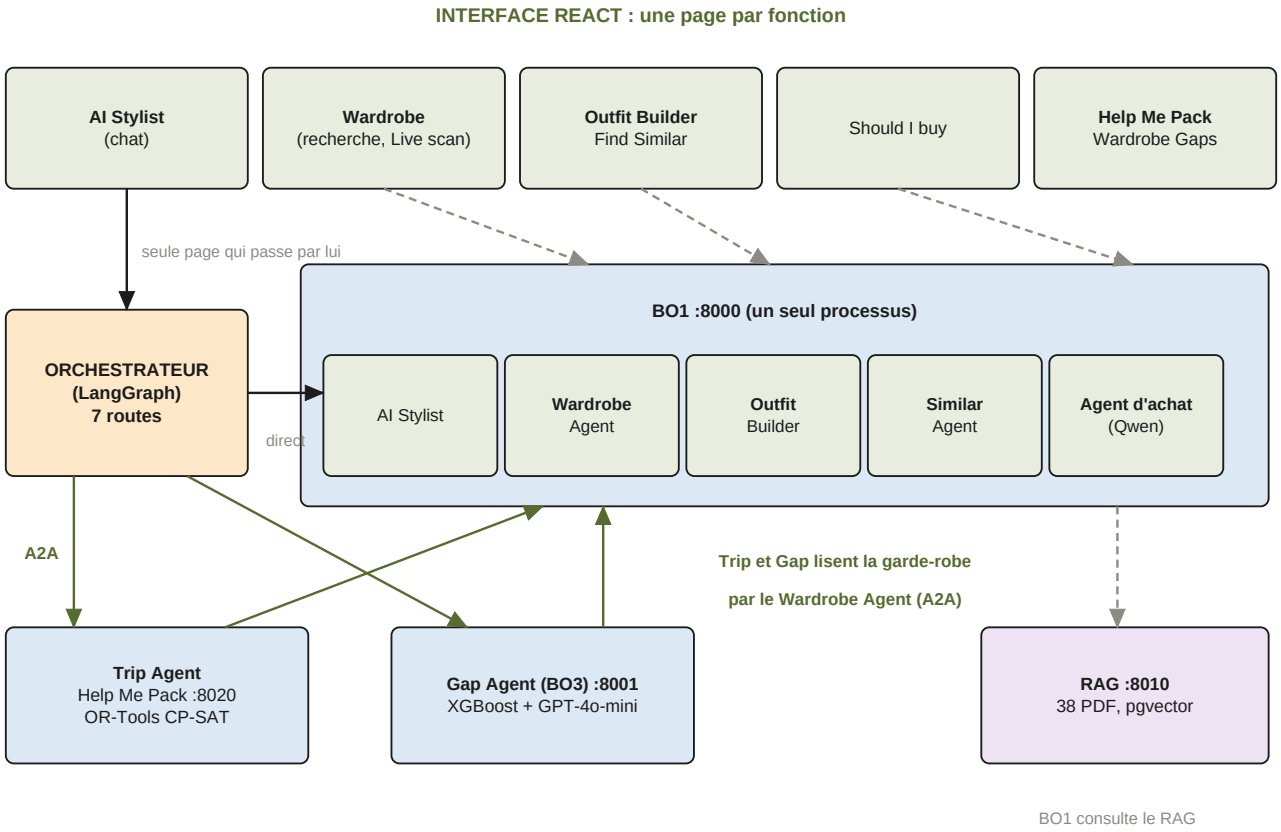

Figure 1 - tous les agents, les pages qui les appellent, et les liens A2A.

1. Les agents, un par un

Agent	Où	Technique	Outils ou nœuds	Sa mission
AI Stylist	BO1 :8000	boucle d'outils GPT-4o, 8 tours	ask_wardrobe_agent, get_weather, fashion_advice	conseiller une tenue avec la vraie météo et les livres de mode
Wardrobe Agent	BO1 :8000	LangGraph : gate, apply, cancel, assist	search_wardrobe, find_similar, wardrobe_stats, update_item (propose)	propriétaire de la garde-robe ; corrige seulement après un oui
Agent anti-doublon	BO1 :8000 (dans le Wardrobe Agent)	boucle d'outils, 4 tours	search_wardrobe	dire si une pièce scannée existe déjà
Outfit Builder	BO1 :8000	LangGraph 5 nœuds	weather, closet, knowledge, compose, validate	3 tenues complètes validées par le code
Similar Agent	BO1 :8000	LangGraph : ask, judge, check	interroge le Wardrobe Agent	pour chaque pièce d'une photo : owned, close ou missing
Agent d'achat	BO1 (dossier smart_purchasing)	Qwen2.5-3B LoRA, 6 tours	analyze_item, find_similar_items, check_outfit_compatibility, analyze_wardrobe_gaps, predict_purchase_ml, calculate_purchase_value	« Should I buy this ? » : BUY, CONSIDER ou SKIP ; sans HF_TOKEN, repli déterministe
Trip Agent	Help Me Pack :8020	GPT-4o + 2 graphes LangGraph + CP-SAT	10 outils (trip, météo, garde-robe, optimiseur, checklist...)	valise du plus petit nombre de pièces qui habille chaque jour
Gap Agent (BO3)	BO3 :8001	boucle d'outils GPT-4o-mini, 6 étapes + XGBoost	5 outils	ce qui manque dans la garde-robe pour un objectif
Orchestrateur	BO1 :8000	LangGraph : route, un nœud par agent, respond	7 routes	choisir l'agent du chat AI Stylist

Ce qui n'est pas un agent

- **Catalogage vision** : YOLO11m-seg puis GPT-4o (JSON strict). Pipeline fixe, le LLM ne choisit pas ses outils.
- **RAG** (:8010) : recherche hybride puis réponse avec sources numérotées [S#]. Appelé comme un outil par l'AI Stylist, l'Outfit Builder, le Wardrobe-fashion et l'orchestrateur de l'équipe.
- **Contrôle qualité et doublon visuel du Live scan** : du code (histogramme et forme), sans LLM.

Une pile séparée de l'équipe, non branchée à l'interface React

Le Stylist BO4 (LangGraph à 9 nœuds, Ollama qwen2.5:3b, :8061) et son orchestrateur (:8070, routes stylist, garde-robe, achat, RAG, et modules externes déclarés dans `services.yaml`) ont leur propre page web. Ils ne parlent pas A2A : ils appellent les services par leurs routes HTTP. À ne pas confondre avec l'orchestrateur de l'application décrit plus bas.

2. Page par page : qui appelle quoi

C'est le tableau qui montre où l'orchestrateur intervient : une seule ligne.

Page	Route appelée	Agent qui répond	Orchestrateur ?
AI Stylist (chat)	`POST /api/assistant/chat`	celui que l'orchestrateur choisit	**OUI**
Wardrobe : recherche IA, corrections	`POST /api/wardrobe/agent`	Wardrobe Agent	non
Wardrobe : Live scan	`quick-detect, analyze, reclassify, check-duplicate`	détecteur YOLO, puis agent anti-doublon	non
Wardrobe : recherche par photo	`POST /api/search/photo`	YOLO, sans LLM	non
Outfit Builder	`POST /api/outfits/agent`	Outfit Builder	non
Find Similar	`POST /api/similar/agent`	Similar Agent (puis boutiques BO3)	non
Should I buy this	`POST /api/shopping-assistant/evaluate`	agent d'achat (Qwen)	non
Help Me Pack	`POST :8020/api/trips`	Trip Agent	non
Help Me Pack : Gap analysis	`POST :8020/api/trips/{id}/gap-analysis`	Trip Agent puis Gap Agent (A2A)	non
Wardrobe Gaps et carte du tableau de bord	`GET /api/bo3/gaps`	Gap Agent (proxy Vite vers :8001)	non

La route `/api/agent/search` (AI Stylist en question libre) existe côté serveur, mais aucune page ne l'appelle aujourd'hui : le chat de l'AI Stylist passe par l'orchestrateur.

Ce que fait l'orchestrateur dans le chat

Route choisie	Quand	Comment il l'appelle
stylist	que porter, couleurs, tissus	appel direct (même processus)
wardrobe	ce que je possède, compter, corriger	appel direct (même processus)
outfit	tenues pour une occasion	appel direct (même processus)
trip	valise, voyage	**A2A** vers :8020 avec le jeton de l'utilisateur
gap	ce qui manque, quoi acheter	**A2A** vers :8001 avec le jeton de l'utilisateur
redirect	« Should I buy » et « Find Similar » : il faut une photo	aucun agent : la réponse ouvre la bonne page
clarify	demande vague ou salutation	aucun agent : pose une question

- **Choix de la route** : le LLM répond en JSON et ne peut citer que les agents de la liste ; des règles FR/EN (valise, manque, tenue...) servent de filet quand il est absent ou se trompe.
- **Mémoire** : un fil par utilisateur et par conversation (checkpointer). Une question laissée ouverte par le Trip Agent (*input-required*) ou une correction qui attend un oui repart au **même agent** au message suivant, quels que soient les mots.

3. Comment les agents communiquent

Trois mécanismes seulement, du plus simple au plus ouvert.

Mécanisme	Utilisé pour	Pourquoi
1. Appel Python direct	orchestrateur vers AI Stylist, Wardrobe Agent, Outfit Builder (même processus BO1)	le plus rapide, aucun réseau
2. A2A (JSON-RPC sur HTTP)	dès qu'un agent en appelle un autre en dehors de son processus, et pour lire la garde-robe	carte publique, jeton de l'utilisateur, tâche avec un état, reprise de conversation par contextId
3. HTTP simple (REST)	page vers services ; BO1 vers RAG (:8010) ; agents vers Open-Meteo	données et services qui ne sont pas des agents

Tous les liens A2A

Appelant	Appelé	Pour quoi	Si l'appel échoue
Orchestrateur	Trip Agent (:8020 /a2a/trip)	planifier une valise	message « not reachable », jamais une erreur 500
Orchestrateur	Gap Agent (:8001 /a2a/gap)	ce qui manque	même message clair
Trip Agent	Wardrobe Agent (:8000 /a2a/wardrobe)	lire la garde-robe (`list`)	repli sur la route BO1 `/api/items`
Trip Agent	Gap Agent (:8001)	bouton « Gap analysis »	erreur affichée dans la page
Gap Agent	Wardrobe Agent	lire la garde-robe	repli sur Supabase
AI Stylist	Wardrobe Agent	`ask` : chercher en mots simples	repli sur sa propre recherche
Similar Agent	Wardrobe Agent	interroger la garde-robe pour chaque pièce	décision par le classement de similarité
Outfit Builder, routes /api/*	Wardrobe Agent	lire la garde-robe (`list`)	lecture directe de la base

Le Wardrobe Agent est ainsi le **point central des données** : tout le monde lit la garde-robe par lui. Aucun autre agent n'a de droit d'écriture sur les vêtements.

Ce qui voyage dans chaque appel

- **Le jeton Supabase de l'utilisateur** (en-tête `Authorization: Bearer`) : chaque agent agit pour lui et ne voit que ses pièces. Il passe dans la configuration du tour, jamais dans l'état du graphe ni dans les traces.
- **contextId** : identifie la conversation ; le rappeler reprend la même tâche (valise en cours, correction en attente).
- **Un état de tâche** : completed, input-required (l'agent pose une question) ou failed.

Protocole A2A en bref

Version 0.3, JSON-RPC 2.0 : `message/send`, `tasks/get`, `tasks/cancel`. Chaque agent publie sa carte à `/.well-known/agent-card.json` (nom, compétences, authentification). Le streaming et les notifications push ne sont pas gérés. Voir le PDF « multi-agents et A2A » pour le détail.

4. Trois parcours de bout en bout

A. « Prépare ma valise pour Paris » dans le chat AI Stylist

```
1. La page envoie le message à /api/assistant/chat avec le jeton.
2. L'orchestrateur choisit trip (LLM + règles).
3. Il envoie une tâche A2A au Trip Agent. Les dates manquent : tâche input-required, « quelles
dates ? ».
4. Tu réponds « du 3 au 7 juin, 8 kg » : aucun mot-clé, mais la question est restée ouverte, donc
le message repart au Trip Agent avec le même contextId.
5. Le Trip Agent lit la garde-robe auprès du Wardrobe Agent (A2A), calcule la météo, optimise
(CP-SAT) et répond.
6. La réponse porte un bouton « Open Help Me Pack » sur la même session de voyage.
```

B. Page Help Me Pack, sans orchestrateur

```
1. La page appelle directement :8020/api/trips avec le jeton.
2. Le Trip Agent lit la garde-robe en A2A (Wardrobe Agent), puis planifie.
3. Le bouton « Gap analysis » appelle le Trip Agent, qui interroge le Gap Agent en A2A.
```

C. Live scan, sans orchestrateur

```
1. La page envoie une image toutes les 600 ms à quick-detect (YOLO seul).
2. L'objet immobile 1,4 s : analyze?fast=true contrôle la qualité et le doublon visuel, puis
sauvegarde.
3. La page appelle l'agent anti-doublon et reclassify (GPT-4o) en arrière-plan.
```

Garde-fous communs à tous les agents

- Le LLM propose et rédige ; le code vérifie, calcule et écrit. Aucun agent ne peut citer une pièce absente de la garde-robe, inventer la météo ou une source.
- Une seule voie d'écriture sur les vêtements, validée par le code, et seulement après un oui de l'utilisateur.
- Chaque appel A2A a un délai maximum ; un service arrêté donne un message clair.
- Boucles d'outils bornées (4 à 8 tours) dans une brique commune, `agent_kit/tool_loop.py`.

Limites à connaître

- Le routage de l'orchestrateur repose sur un LLM : une demande ambiguë peut finir en « clarify ».
- Chaque saut A2A ajoute un appel réseau et souvent un appel LLM : plus précis, mais plus lent et plus cher.
- Les tâches A2A et les propositions de correction sont gardées en mémoire du processus : un redémarrage les efface.
- L'agent d'achat dépend de Hugging Face (HF_TOKEN) ; sans lui, il n'est pas un agent mais une évaluation déterministe.